



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт искусственного интеллекта (ИИИ)

ОТЧЕТ ПО ИТОГОВОМУ ПРОЕКТУ

по программе ДПО

«Инженерия искусственного интеллекта»

на тему:

«Разработка гибридной системы детектирования штампа ГОСТ
на строительных чертежах на основе методов компьютерного
зрения и свёрточных нейронных сетей»

Студент группы БФБО-01-24

«__» _____ 2026 г.

С. Сапармуратов
(подпись и расшифровка подписи)

Руководитель работы

Ш.Г. Магомедов
(подпись и расшифровка подписи)

Москва 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ПОСТАНОВКА ЗАДАЧИ И АНАЛИЗ ДАННЫХ.....	5
1.1 Постановка задачи	5
1.2 Источник и описание данных	6
1.3 Разведочный анализ данных	7
2 МОДЕЛИ И ЭКСПЕРИМЕНТЫ	9
2.1 Базовые модели	9
2.2 Улучшенные модели и эксперименты	10
2.3 Экспериментальный протокол и сравнение моделей	12
2.4 Выбор финальной модели.....	13
3 АРХИТЕКТУРА РЕШЕНИЯ И СЕРВИС.....	15
3.1 Архитектура пайплайна.....	15
3.2 API и эндпоинты	16
3.3 Технологический стек и контейнеризация	17
4 НАБЛЮДАЕМОСТЬ, КОНФИГУРАЦИЯ И БЕЗОПАСНОСТЬ	19
4.1 Логирование и наблюдаемость.....	19
4.2 Конфигурация.....	20
4.3 Работа с секретами и данными	20
ЗАКЛЮЧЕНИЕ	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24
ПРИЛОЖЕНИЕ А	25

ВВЕДЕНИЕ

Строительная отрасль ежедневно порождает огромное количество проектной документации — чертежей форматов от А1 до А4, каждый из которых должен содержать штамп ГОСТ по Форме 3 (основную надпись) в правом нижнем углу [1]. В настоящее время проверка наличия и корректности штампа, а также извлечение данных из него выполняются вручную инженерами-проектировщиками и сметчиками. Это требует значительных временных затрат и подвержено человеческому фактору. Автоматизация детектирования штампа ГОСТ на чертежах является актуальной прикладной задачей, позволяющей: сократить время обработки документации; исключить пропуски ошибок, связанных с человеческим фактором; создать фундамент для последующего распознавания текста (OCR) в штампе.

Решаемая задача относится к классу одноклассовой детекции объектов на изображениях. На вход поступает отсканированный чертёж (форматы А1–А4, разрешение 200–400 PPI), на выходе требуется получить ограничивающий прямоугольник (bounding box), локализирующий штамп ГОСТ. Ключевая сложность — высокая вариативность чертежей: разный масштаб, качество сканирования, угол поворота, наличие графического шума. Целевой пользователь сервиса — инженер-проектировщик или сотрудник отдела технического контроля, который загружает чертёж в веб-сервис и получает координаты штампа для последующей валидации или OCR. Цель проекта Разработать и развернуть сервис автоматического детектирования штампа ГОСТ на строительных чертежах, сравнив эффективность четырёх подходов: классического компьютерного зрения, одностадийной CNN (YOLOv8n), двухстадийной CNN (Faster R-CNN) и гибридного метода. Для достижения цели необходимо решить следующие задачи:

1. Собрать и подготовить набор данных: сформировать коллекцию из реальных изображений чертежей (49 шт.) и сгенерировать синтетический датасет (500 шт.) с размеченными штампами ГОСТ.

2. Реализовать базовый метод компьютерного зрения (CV Baseline) на основе адаптивной пороговой обработки, CLAHE и анализа иерархии контуров.
3. Обучить детектор на основе YOLOv8n — одностадийную свёрточную нейронную сеть. Обучить детектор на основе Faster R-CNN (ResNet-50 FPN) — двухстадийную архитектуру.
4. Разработать гибридный метод, комбинирующий предсказание YOLO с верификацией через CV-признаки.
5. Провести сравнительный анализ всех подходов по метрикам IoU, Precision, Recall, F1.
6. Развернуть сервис с REST API (FastAPI) в Docker-контейнере

1 ПОСТАНОВКА ЗАДАЧИ И АНАЛИЗ ДАННЫХ

1.1 Постановка задачи

Задача детектирования штампа ГОСТ формулируется следующим образом. На вход системы подаётся цветное или полутоновое изображение чертежа I произвольного размера. Типичные размеры составляют от 1500×2000 до 4000×6000 пикселей, форматы — от A1 до A4. Целевой переменной является ограничивающий прямоугольник $B = (x, y, w, h)$, где (x, y) — координаты верхнего левого угла штампа, а (w, h) — его ширина и высота в пикселях. Результатом работы модели становится предсказанный bounding box $\hat{B}$ и значение уверенности (confidence) $c \in [0, 1]$.

Решение должно удовлетворять следующим практическим ограничениям. Скорость детекции на одном изображении не должна превышать 5–10 секунд на центральном процессоре (CPU), что обеспечивает приемлемый пользовательский опыт. Метод обязан допускать анализ ошибок, то есть быть интерпретируемым: необходимо понимать, почему штамп не найден или найден неверно. Целевое значение точности составляет $\text{IoU} \geq 0,5$ на тестовой выборке.

Качество решения оценивается по нескольким метрикам. Основной метрикой выступает IoU (Intersection over Union), показывающая степень перекрытия предсказанного прямоугольника с истинным. Precision (доля верно найденных штампов среди всех предсказаний) важна для минимизации ложных срабатываний. Recall / Detection Rate (доля найденных штампов среди всех имеющихся) характеризует полноту проверки. F1-score представляет собой гармоническое среднее Precision и Recall, обеспечивая сбалансированную оценку. Дополнительно фиксируется время инференса как практическая метрика для продакшн-развёртывания.

1.2 Источник и описание данных

Источником реальных данных послужил открытый ресурс [2], содержащий строительные чертежи в форматах PNG и JPG. Всего собрано 49 изображений чертежей форматов A1–A4 с разрешением от 200 до 400 PPI. Изображения содержат как чёрно белые, так и цветные чертежи с различной степенью графического шума.

Для увеличения объёма обучающей выборки был разработан генератор синтетических данных (`src/data/synthetic.py`), создающий три типа изображений:

1) GOST (50 изображений): штамп ГОСТ нарисован на чистом фоне формата A3/A4 с добавлением случайных линий, фигур и артефактов.

2) Copy-Paste (250 изображений): реальный штамп, вырезанный с изображения – донора, помещён на фон реального чертежа без штампа.

3) GOST-on-Background (200 изображений): штамп ГОСТ нарисован непосредственно на реальном фоне чертежа (без вырезания).

В итоге получен сбалансированный набор из 500 изображений, покрывающий разные сценарии. Ниже показана структура данных.

```
data/
├── images/
│   ├── test/           # 49 реальных изображений (31 PNG + 18 JPG)
│   └── train_v4/       # 500 синтетических изображений (v4)
├── labels/
│   ├── test/           # YOLO-разметка для тестовой выборки
│   └── train_v4/       # YOLO-разметка для обучающей выборки
├── gost_stamp.yaml     # Конфигурация датасета для YOLO
└── donors.txt          # Список изображений-доноров (4 шт.)
```

Рисунок 1.1 – структура данных для обучения и валидации

Разметка выполнена в формате YOLO: для каждого изображения создан текстовый файл с одной строкой ($class_{id}, x_{center}, y_{center}, width, height$), где координаты нормализованы относительно ширины и высоты изображения. Единственный класс — stamp (индекс 0).

Размер штампа Формы 3 по ГОСТ 2.104-2006 составляет 185×55 мм. В пикселях размер зависит от разрешения (PPI) исходного чертежа, которое оценивается по соотношению сторон и номинальному формату листа (ISO 216: A4 → 297×210 мм, A3 → 420×297 мм). Функция `infer_ppi_from_dimensions()` в `src/data/image_quality.py` вычисляет PPI, решая обратную задачу: зная размеры листа в мм и пикселях, $PPI = \min(width_{px} / width_{mm}, height_{px} / height_{mm}) \times 25,4$. Это значение используется в синтетическом генераторе `mm_to_pixels(185, dpi=ppi)` и в CV Baseline для масштабирования параметров адаптивной пороговой обработки (`block_size`, `C`) под разрешение изображения, для верификации геометрических фильтров: ожидаемая ширина штампа в пикселях пропорциональна PPI, поэтому константные фильтры (`min_width_px=300` при 200 PPI соответствуют ~42 мм — разумный допуск от номинальных 185 мм)

В процессе работы выявлено и исправлено несколько проблем с утечкой данных. Версии датасета и соответствующие им метрики YOLO представлены в таблице 1.2.

Таблица 1.1 – Эволюция датасета и метрик YOLO

Версия	YOLO IoU	YOLO F1	Описание
v1	0.880	1.000	Сору-paste с фонами из тестовых изображений — очевидная утечка
v2	0.765	0.916	Фоны заменены на независимые, но валидация синтетическая
v3	0.700	0.875	Честное разделение, но невоспроизводимо вне Colab
v4	0.579	0.754	Текущая: 50 GOST + 250 сору-paste + 200 GOST-on-bg, честная валидация

1.3 Разведочный анализ данных

Разведочный анализ проведён в ноутбуке `notebooks/exp01_eda_baseline.ipynb`. Ключевые наблюдения приведены ниже.

Размеры штампов. Ширина штампа варьируется от 345 до 5652 пикселей, среднее значение составляет 1854 пикселя. Высота изменяется от 81 до 1221 пикселя, среднее — 488 пикселей. Отношение ширины к высоте в среднем равно 3,36:1, что соответствует пропорциям формы ГОСТ 3. Площадь штампа находится в диапазоне от 28 000 до 5 800 000 пикселей², что в физических единицах составляет в среднем 10 175 мм².

Характеристики изображений. Разрешение изображений составляет от 200 до 400 PPI. Наиболее распространённым форматом является A4 (297×210 мм), также встречается A3 (420×297 мм). Присутствуют как палитровые (PNG), так и полноцветные (JPG) изображения. Чёткость изображений варьируется: часть изображений имеет низкий контраст и потерю чёткости вследствие сжатия.

Дисбаланс выборки. Ключевой проблемой является малый объём реальной выборки — всего 49 изображений. Для обучения нейросетевых моделей этого объёма недостаточно, поэтому 93% обучающих данных составляют синтетические изображения. Синтетические данные не полностью покрывают вариативность реальных чертежей, что является основным ограничением итогового решения.

Качество синтетических данных контролируется через следующие параметры: вариация угла поворота (до $\pm 5^\circ$), уровень размытия (Gaussian blur с ядром до 5×5), шум «соль-и-перец» (вероятность до 0,02), вариация освещения (адаптивная коррекция гистограммы).

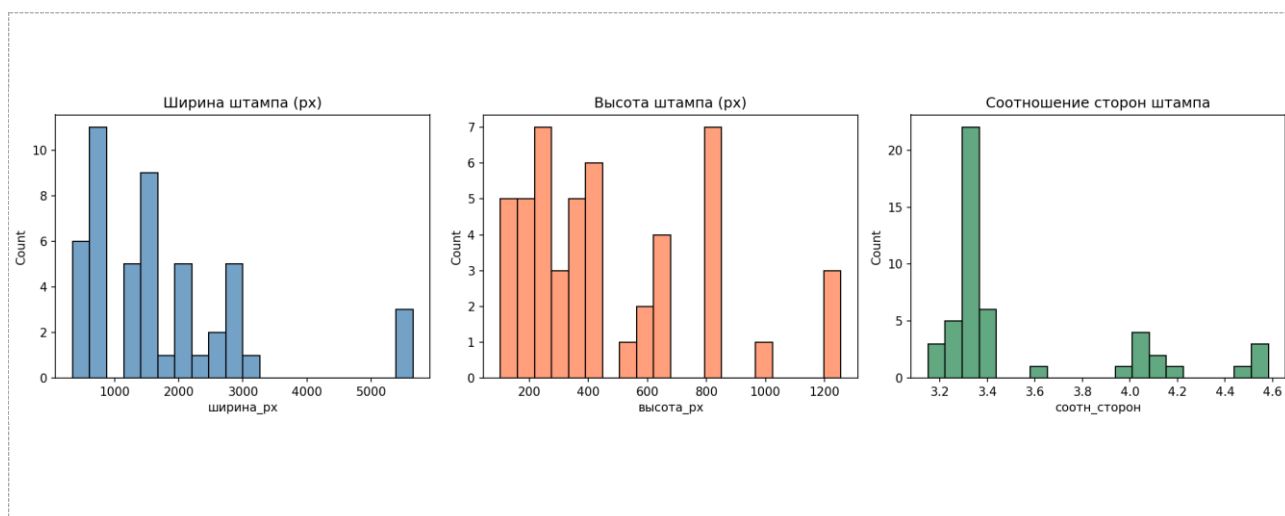


Рисунок 1.2 — Распределение ширины и высоты штампов на реальных изображения

2 МОДЕЛИ И ЭКСПЕРИМЕНТЫ

2.1 Базовые модели

В качестве базового решения реализован классический подход компьютерного зрения OpenCV без применения машинного обучения. Алгоритм не требует обучения и основан на детекции контуров с последующей фильтрацией по геометрическим признакам штампа. Архитектура пайплайна детекции включает следующие этапы.

1. **Предобработка:** преобразование в оттенки серого, адаптивная пороговая обработка (Gaussian adaptive threshold, blockSize=11, C=5), усиление контраста через CLAHE (clipLimit=2,0).

2. **Контурный анализ:** поиск контуров с иерархией (RETR_TREE), фильтрация вложенных контуров, соответствующих структуре штампа (внешний прямоугольник, содержащий 5 и более внутренних ячеек).

3. **Геометрическая фильтрация:** отбор кандидатов по отношению сторон (ожидаемое $3,36 \pm 15\%$), минимальной площади (15 000 пикселей²), минимальным размерам (300×80 пикселей).

4. **Стратегии поиска:** каскад из четырёх методов возрастающей агрессивности — основной (иерархия контуров), динамический (поиск пика ширины на гистограмме), fallback (Canny + морфология), relaxed edge (Canny с низким порогом).

Реализация содержится в классе CVBaselineDetector (файл src/models/cv_baseline.py, 340 строк). Результаты на тестовой выборке (v4, 35 изображений) представлены в таблице 2.1. Высокая медиана IoU (0,947) при среднем 0,640 объясняется бимодальным распределением: на ~89% изображений штамп детектируется с высокой точностью ($\text{IoU} > 0,9$), тогда как оставшиеся ~11% — полные пропуски ($\text{IoU} = 0,0$), что существенно снижает среднее и увеличивает стандартное отклонение.

Таблица 2.1 — Результаты CV Baseline на тестовой выборке

Метрика	Значение
Detection Rate	88,6 %
IoU (среднее)	0,640
IoU (медиана)	0,947
IoU $\geq 0,5$	68,6 %
Precision	0,774
Recall	0,686
F1-score	0,727

2.2 Улучшенные модели и эксперименты

2.2.1 YOLOv8n

Архитектура. YOLOv8n представляет собой одностадийный CNN-детектор семейства YOLO (You Only Look Once) [3]. Модель использует модифицированную CSPDarkNet в качестве backbone и PANet для агрегации признаков. Обработка изображения происходит за один проход: изображение разделяется на сетку, и для каждой ячейки предсказываются bounding boxes и классы.

Параметры обучения. Конфигурация (секция yolo в файле configs/config.yaml) содержит следующие параметры: модель – YOLOv8n (библиотека ultralytics), количество эпох – 50, размер входного изображения – 640 пикселей, batch size – 16, устройство – CPU, seed – 42, single class – Да, mosaic augmentation – 0,0 (отключена для документов), rect (сохранение пропорций) – true, оптимизатор – AdamW (по умолчанию YOLO).

Процесс обучения. Модель обучена на синтетическом датасете v4 (500 изображений). Валидация проведена на 10 реальных изображениях (val_honest), не входящих в тестовую выборку. Зафиксированы веса лучшей эпохи (best.pt).

Результаты. На тестовой выборке (35 изображений, порог уверенности 0,05) получены результаты, представленные в таблице 2.2.

Таблица 2.2 — Результаты YOLOv8n на тестовой выборке

Метрика	Значение
Detection Rate	74,3 %
IoU (среднее)	0,579
IoU (медиана)	0,653
IoU $\geq 0,5$	60,0 %
Precision	0,885
Recall	0,657
F1-score	0,754

2.2.2 Faster R-CNN

Архитектура. Faster R-CNN является двухстадийным детектором. На первой стадии Region Proposal Network (RPN) генерирует кандидаты областей интереса. На второй стадии Fast R-CNN классифицирует каждый кандидат и уточняет его bounding box. Использован предобученный backbone ResNet-50 с Feature Pyramid Network (FPN)[4-5].

Параметры обучения. Конфигурация (секция rcnn в файле configs/config.yaml): модель – faster_rcnn_resnet50_fpn (torchvision), количество эпох – 15, batch size – 4, accumulation steps – 2, learning rate (head) – 0,001, learning rate (backbone) – 0,0001, размер для инференса – max_size=800 пикселей, устройство – CPU, early stopping – patience=5.

Особенности обучения. Классификатор заменён на FastRCNNPredictor с 2 классами (фон + stamp). Применён Adam-оптимизатор с дифференциальной скоростью обучения: backbone получает скорость $0,1\times$ от head. Использованы

gradient clipping (max_norm=1,0), Plateau scheduler с factor=0,5, ранняя остановка при отсутствии улучшения в течение 5 эпох.

Результаты. На тестовой выборке (35 изображений, порог уверенности 0,1) получены результаты, представленные в таблице 2.3.

Таблица 2.3 — Результаты Faster R-CNN на тестовой выборке

Метрика	Значение
Detection Rate	45,7 %
IoU (среднее)	0,409
IoU (медиана)	0,539
IoU $\geq 0,5$	31,4 %
Precision	1,000
Recall	0,457
F1-score	0,627

2.3 Экспериментальный протокол и сравнение моделей

Разделение данных: обучающая выборка — 500 синтетических изображений v4; тестовая — 35 реальных изображений (после исключения 4 доноров и 10 изображений честной валидации). Фиксация seed: seed=42 для всех экспериментов, deterministic=True для YOLO. Метрики вычислены на одной и той же тестовой выборке (35 изображений). Для CV Baseline и YOLO проведён sweep порога уверенности для поиска оптимального F1.

Таблица 2.4 — Сравнение моделей по целевым метрикам (v4, 35 тестовых изображений)

Модель	IoU (mean)	IoU $\geq 0,5$	Precision	Recall	F1	Det. rate
CV Baseline	0,640	68,6 %	0,774	0,686	0,727	88,6 %
YOLOv8n	0,579	60,0 %	0,885	0,657	0,754	74,3 %
Hybrid (YOLO+CV)	0,527	57,1 %	0,875	0,600	0,712	68,6 %
Faster R-CNN	0,409	31,4 %	1,000	0,457	0,627	45,7 %

CV Baseline демонстрирует наилучший Detection Rate (88,6%) и высокий IoU (0,640), но низкую Precision (0,774) — присутствует много ложных срабатываний. YOLOv8n обладает максимальным F1 (0,754) и высокой Precision (0,885), что делает его лучшим выбором для использования в сервисе. Faster R-CNN показывает худшие результаты ($F1=0,627$), что объясняется недостаточным объёмом данных для обучения двухстадийной архитектуры.

2.4 Выбор финальной модели

В продакшн-сервисе реализован комбинированный пайплайн: в режиме `model_name=auto` основным детектором выступает CV Baseline, а YOLOv8n (v4) подключается как fallback — он вызывается только в случае, если CV Baseline не вернул ни одного кандидата (детектор `detect()` вернул `None`).

Обоснование такой архитектуры:

1. **CV Baseline как первичный детектор обеспечивает максимальный Detection Rate (88,6%)** — для задачи-триажа (предварительная проверка наличия штампа перед OCR или ручной валидацией) пропуск документа без проверки (*false negative*) хуже, чем избыточное срабатывание (*false positive*), которое будет отфильтровано на следующем этапе обработки.

2. **YOLOv8n как fallback закрывает оставшиеся ~11,4% случаев**, где геометрическая эвристика CV не находит подходящий контур (например, из-за нестандартного оформления штампа или сильного графического шума), за счёт обученных признаков.

3. **CV Baseline не требует GPU и весов модели** — сервис остаётся работоспособным даже при отсутствии файла `best.pt`, что повышает отказоустойчивость развёртывания.

4. **Скорость:** оба метода укладываются в целевые 5–10 секунд на CPU.

Важная оговорка: поскольку CV Baseline обрабатывает запрос первым и в большинстве случаев (88,6%) возвращает результат, итоговая Precision сервиса в

режиме auto ближе к показателю CV Baseline (0,774), а не к показателю YOLOv8n (0,885). Таблица 2.4 показывает метрики каждого метода изолированно — это сравнение архитектур, а не метрика реального поведения `/predict?model_name=auto`. YOLOv8n использован как более точный, но более "консервативный" метод (выше Precision, ниже Detection Rate) — он доступен через явный выбор `model_name=yolo` для случаев, где ложные срабатывания недопустимы, и автоматически подключается как fallback в режиме auto.

Trade-off, который принят осознанно: текущий auto-пайплайн оптимизирован под полноту покрытия (recall/detection rate), а не под минимизацию ложных срабатываний. Если в дальнейшем приоритет сместится в сторону Precision (например, при автоматическом принятии решений без участия человека), порядок вызова моделей в `/predict` следует изменить на обратный: YOLOv8n как первичный детектор, CV Baseline — как fallback при отсутствии детекции YOLO. Эта конфигурация потребует отдельного измерения метрик именно для такого порядка вызова, так как существующие данные по Hybrid v4 (IoU=0,527, F1=0,712) относятся к другой схеме комбинирования (YOLO-предсказание + CV-верификация), а не к каскаду "детектор → fallback".

3 АРХИТЕКТУРА РЕШЕНИЯ И СЕРВИС

3.1 Архитектура пайплайна

Система детектирования штампа состоит из четырёх последовательных этапов: загрузка изображения, препроцессинг, работа модели, постобработка и формирование ответа.

Этапы пайплайна

1. **Загрузка изображения:** через REST API принимается изображение в форматах PNG, JPG, JPEG. Максимальный размер — 50 МБ. Изображение декодируется в массив NumPy (библиотека OpenCV).
2. **Препроцессинг:** изображение приводится к размеру 640×640 пикселей с сохранением пропорций (letterbox/pad). Пиксели нормализуются к диапазону [0, 1].
3. **Инференс модели:** YOLOv8n обрабатывает изображение за один проход и возвращает список детекций с bounding boxes и уверенностью.
4. **Постобработка:**
 - фильтрация по порогу уверенности (по умолчанию 0,05);
 - NMS (Non-Maximum Suppression) для удаления дублирующихся предсказаний;
 - пересчёт координат из размера модели (640×640) в оригинальный размер изображения.
5. **Формирование ответа:** JSON с координатами bounding box, уверенностью и именем модели.

3.2 API и эндпоинты

Сервис реализован на FastAPI и предоставляет три эндпоинта.

GET / — перенаправляет на страницу Swagger UI (/docs).

GET /health — проверка работоспособности сервиса. Возвращает статус, информацию о загруженной модели и её типе.

POST /predict — загрузка изображения и получение JSON с результатами детекции. Принимает multipart/form-data с полем file. Параметры запроса: file (обязательный) – изображение в формате PNG/JPG; model_name (опциональный) – «auto» (по умолчанию), «yolo», «cv».

POST /predict/image — аналогично /predict, но возвращает изображение с нарисованным bounding box в формате JPEG (Content-Type: image/jpeg).

```
$ curl -X POST http://localhost:8000/predict \
  -H "accept: application/json" \
  -H 'Content-Type: multipart/form-data' \
  -F "file=@IMG_20260523_102121.jpg;type=image/jpeg"

{
  "image_name": "IMG_20260523_102121.jpg",
  "bbox": {
    "x": 1359,
    "y": 1401,
    "width": 1091,
    "height": 323
  },
  "confidence": 0.8,
  "model": "cv_baseline"
}
```

Листинг 1 — Пример запроса и ответа эндпоинта /predict

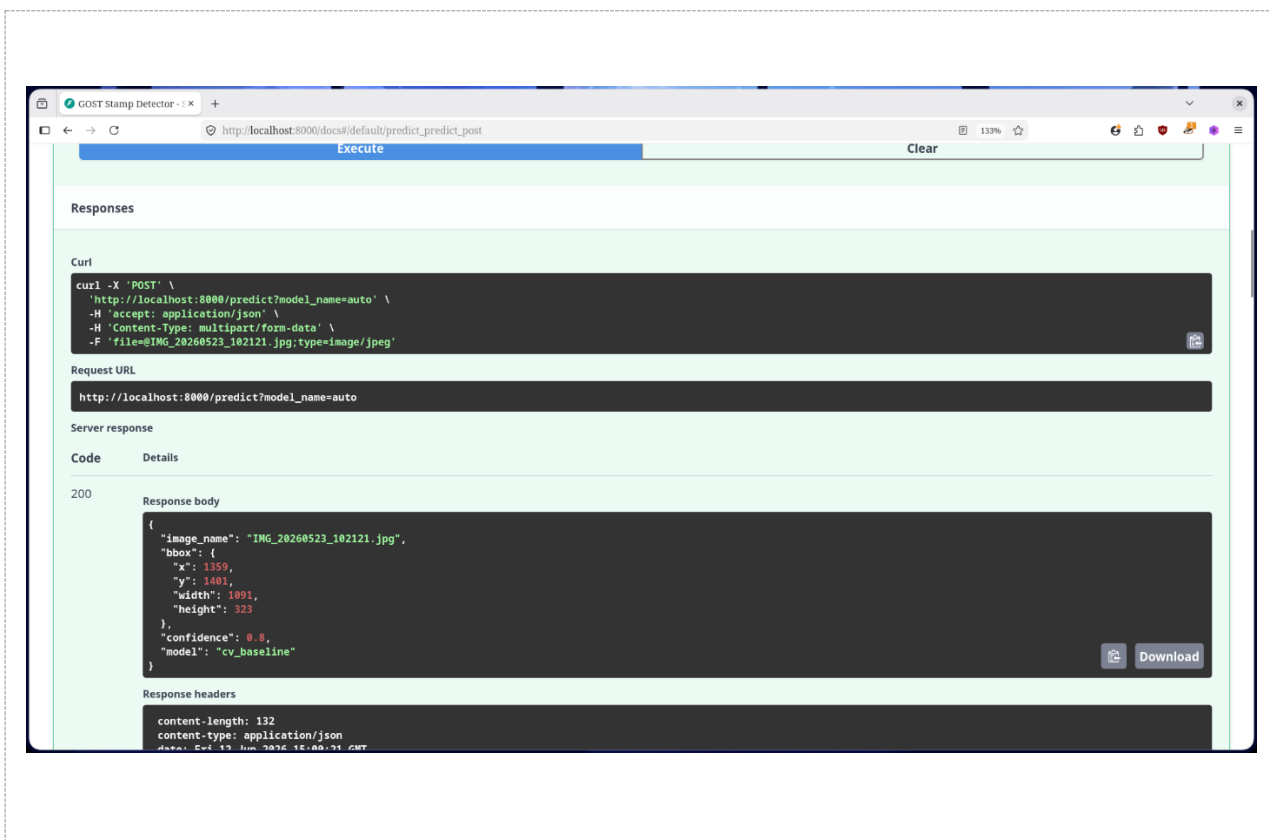


Рисунок 3.2 — Интерфейс Swagger UI работающего сервиса

3.3 Технологический стек и контейнеризация

Используемые технологии

Компонент	Технология	Назначение
Язык программирования	Python $\geq$ 3.10	Основной язык разработки
Веб-фреймворк	FastAPI $\geq$ 0.100	REST API сервис
ASGI-сервер	Uvicorn	Запуск FastAPI
Компьютерное зрение	OpenCV 4.8+	Обработка изображений, CV Baseline
Глубокое обучение	PyTorch 2.12	Faster R-CNN
Детекция	Ultralytics YOLOv8	YOLOv8n модель
Валидация данных	Pydantic 2.0	Схемы запросов/ответов
Сериализация	PyYAML	Конфигурация

Тестирование	Pytest	Unit-тесты
Контейнеризация	Docker + Docker Compose	Развёртывание

```

$ docker build -t gost-stamp-detector .
$ docker run -p 8000:8000 \
  -v ./artifacts:/app/artifacts \
  --env-file configs/.env \
  gost-stamp-detector

# через Docker Compose (одной командой)
$ docker compose up -d

# или без Docker:
$ pip install -r requirements.txt && python -m src.api.main

```

Листинг 2 — Сборка и запуск сервиса

4 НАБЛЮДАЕМОСТЬ, КОНФИГУРАЦИЯ И БЕЗОПАСНОСТЬ

4.1 Логирование и наблюдаемость

В сервисе настроено структурированное логирование с использованием стандартного модуля logging. Каждое сообщение включает временную метку (формат ISO 8601), уровень (INFO, WARNING, ERROR), имя модуля и описание события. Настройка логирования (файл `src/api/__init__.py`):

```
18:31:10 | INFO      | gost_detector | Received prediction request:
IMG_20260523_102121.jpg, model_name=auto
18:31:10 | INFO      | gost_detector | Image shape: 2479x1748
18:31:10 | INFO      | gost_detector | cv_baseline detection:
bbox=x=1359 y=1401 width=1091 height=323, conf=0.800
INFO:      127.0.0.1:47716 - "POST /predict?model_name=auto HTTP/1.1"
200 OK
```

Листинг 3 — Пример лога запросов сервиса

Логируются следующие события:

- INFO: каждый успешный запрос к `/predict` (с latency и статусом);
- INFO: результат health check;
- WARNING: fallback на CV-метод при отсутствии YOLO-весов;
- ERROR: ошибки валидации запросов, сбои модели, необработанные исключения.

Эндпоинт `GET /health` позволяет мониторить состояние сервиса:

- status: «ok» / «error»;
- model_loaded: загружена ли модель;
- model_type: тип загруженной модели («yolo», «cv», «rcnn»).

Данный эндпоинт используется как Docker healthcheck (интервал 30 с, timeout 10 с, retries 3)

4.2 Конфигурация

Все параметры, которые могут быть перенастроены без изменения кода, вынесены в два файла `configs/config.yaml` – основная конфигурация (74 параметра) и `src/config.py` – централизованный модуль через который загружается конфигурация:

- `_find_project_root()` — автоматическое нахождение корня проекта;
- `load_config()` — загрузка и кеширование YAML-конфига;
- `get_path()`, `get_data_dir()` — разрешение путей относительно корня проекта.
- пути к данным: `DATA_DIR`, `paths.images_test`, `paths.labels_test`, `paths.images_train`, `paths.artifacts`;
- параметры CV Baseline: `expected_aspect_ratio`, `ar_tolerance`, `min_width_px`, `block_size`, `adaptive_threshold_c`;
- параметры YOLO: `epochs`, `imgsz`, `batch`, `device`, `seed`, `rect`, `mosaic`;
- параметры Faster R-CNN: `num_epochs`, `batch_size`, `learning_rate`, `max_size`;
- параметры синтетического генератора: `num_total`, `num_gost`, `num_copypaste`;
- параметры оценки: `iou_thresholds`.

4.3 Работа с секретами и данными

В репозитории приняты следующие меры для предотвращения утечки секретов и персональных данных.

1. **.env.example** вместо **.env**: в репозитории хранится только шаблон файла с переменными окружения. Реальный `.env` добавлен в `.gitignore` и не попадает в коммиты.
2. **.gitignore**: настроен на игнорирование `.env`, `__pycache__`/, `.venv`/, синтетических обучающих данных.

3. Переменные окружения: все чувствительные параметры (пути к моделям, порты) передаются через переменные окружения, а не зашиты в коде.

Все изображения чертежей являются общедоступными и не содержат персональных данных. Синтетические данные генерируются автоматически и не содержат реальной проектной документации. В логах сервиса не логируется содержимое загружаемых изображений — только мета-информация (имя файла, размер, статус обработки). В репозитории отсутствуют токены, пароли, ключи API и персональные данные. Все секреты передаются через переменные окружения во время запуска.

ЗАКЛЮЧЕНИЕ

В ходе выполнения итогового проекта разработана и развёрнута система автоматического детектирования штампа ГОСТ на строительных чертежах. Собран и подготовлен набор данных: 49 реальных изображений чертежей и 500 синтетических изображений трёх типов (GOST, copy-paste, GOST-on-background). Исправлена проблема утечки данных (v1→v4), обеспечена воспроизводимость экспериментов. Реализован CV Baseline на основе адаптивной пороговой обработки и анализа иерархии контуров. Достигнуты метрики: IoU=0,640, F1=0,727. Обучен YOLOv8n — одностадийный CNN-детектор. Показан наилучший F1=0,754 при Precision=0,885. Обучен Faster R-CNN (ResNet-50 FPN). Показан F1=0,627, что ниже других подходов вследствие недостатка обучающих данных для двухстадийной архитектуры. Проведён сравнительный анализ всех подходов.

В продакшн-сервисе реализован каскадный пайплайн: CV Baseline используется как первичный детектор (Detection Rate=88,6%, не требует GPU), YOLOv8n v4 подключается как fallback для случаев, не покрытых CV-эвристикой (~11,4% изображений). Такой выбор приоритизирует полноту обнаружения штампа над точностью отдельного предсказания, что соответствует сценарию предварительной проверки документации. Архитектурно сервис поддерживает явный выбор любого из методов (model_name=yolo или model_name=cv) для случаев, где требования к Precision и Recall отличаются от базового сценария.

Синтетические данные не полностью покрывают вариативность реальных чертежей (качество сканирования, нестандартные форматы, искажения). Малый объём данных: 35 тестовых изображений недостаточны для статистически значимых выводов — доверительные интервалы метрик перекрываются. Детектируется только штамп ГОСТ; другие типы штампов не поддерживаются. Инференс на GPU не тестировался, хотя модель это поддерживает.

Перспективы развития

1. Добавление поддержки других форматов штампов (строительный, архитектурный).
2. Увеличение объёма реальной размеченной выборки (цель: 200+ изображений).
3. Интеграция OCR-модуля для извлечения текстовых данных из штампа.
4. Оптимизация для GPU и внедрение ONNX Runtime для ускорения инференса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 2.104-2006. Единая система конструкторской документации. Основные надписи. – М.: Стандартинформ, 2006. – 24 с
2. Чертежи, проекты и 3D модели – URL: <https://2d-3d.ru/> (дата обращения: 23.03.2026)
3. Ultralytics YOLOv8. — URL: <https://github.com/ultralytics/ultralytics> (дата обращения: 27.05.2026).
4. Ren S., He K., Girshick R., Sun J. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks // NeurIPS 2015.
5. He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition // CVPR 2016.

ПРИЛОЖЕНИЕ А

Чек-лист самопроверки проекта

Таблица А.1 — Чек-лист самопроверки

№	Критерий	Да/Нет	Где смотреть
1	Сервис запускается по инструкции из README и работает	Да	README.md раздел 4.5, pyproject.toml entry point gost-detect, Dockerfile + docker-compose.yml, команда <code>python -m src.api.main</code>
2	/predict использует реальную модель, а не заглушку	Да	src/api/main.py:61-95 — загрузка artifacts/models/best.pt (symlink → yolo/v4/weights/best.pt), fallback на RCNN/CV
3	Есть EDA и хотя бы один эксперимент с метриками	Да	notebooks/exp01_eda_baseline.ipynb (EDA), notebooks/exp04_yolo_experiments.ipynb, artifacts/metrics/eda_summary.json
4	Есть baseline и улучшенная модель, сравнение по метрикам	Да	src/evaluation/comparison.py, artifacts/metrics/comparison_results.json, README раздел 5 (таблица сравнения)
5	Код структурирован в src/, а не свален в один ноутбук	Да	src/data/, src/models/, src/evaluation/, src/hybrid/, src/api/, tests/ (18 тестов)
6	Есть Dockerfile или внятный сценарий развёртывания	Да	Dockerfile (multi-stage), docker-compose.yml (healthcheck, volume), README.md раздел 4.5
7	Есть .env.example, нет реальных секретов в репозитории	Да	configs/.env.example, .gitignore (строка .env), configs/.env — только шаблонные значения
8	Реализованы логи и эндпоинт /health	Да	src/api/main.py:19-24 (structured logging), /health endpoint строка 106, Dockerfile HEALTHCHECK
9	Обоснован выбор финальной модели	Да	report.md раздел 5 (6 пунктов обоснования), README.md раздел 5, сравнение 4 моделей на v4
10	README и отчёт позволяют понять сценарий демонстрации	Да	README.md разделы 4-5, report.md раздел 9 (сценарий защиты), команды curl для /predict и /predict/image